

Live Demonstration Guide

MARL for Cooperative Payload Transport & Collision Avoidance

Presenting to: Dr. S. S. Ohl, COEP Technological University **One rule for the whole demo:** *you never type — you only double-click and press numbers.*

1. What you are showing (30-second pitch, memorize this)

*"Three rovers must drag a shared payload to a goal zone using only local sensors and a shared team reward. Nothing about cooperation is programmed — the coordination **emerges from training**. A random policy scores 0%. My QMIX team delivers the payload **50% of the time**, roughly **twice as fast** as a MAPPO team, using only 9 LiDAR beams per rover. A classical potential-fields controller does reach 100% — but only because it cheats with privileged ground-truth sensing. I'll show you all four live."*

2. The night before (10 minutes)

1. Double-click `RUN_DEMO.bat` — the menu appears.
2. Press **3** (QMIX). Press **R** two or three times inside the window. Confirm rovers move purposefully toward the goal. Press **ESC**.
3. Press **4** (MAPPO). Watch ~30 seconds. Press **ESC**.
4. Press **5** — confirm the approved project PPT ("topic of project.pptx") opens and displays fine.
5. Press **7** — let the 30-episode benchmark run once (~3 min) so the console prints the fresh results table. (Optional but great: you can say "I just re-ran the formal evaluation in front of you.")
6. Close everything. Plug the laptop charger in before the meeting.

3. Ten minutes before

- Open `RUN_DEMO.bat` and leave the menu on screen (black console).
- Press **8** once to open the plots folder in the background (minimize it) — instant fallback material.
- Windows power plan: "Best performance" (Settings → Power), so PyGame runs at full speed.
- Close browsers/chat apps. Set screen scale to 100% if the PyGame window looks too big.

4. THE FLOW — 90-second short version

If time is short, this is the whole demonstration:

Step	Press	What happens	What you say
1	1	Random-policy window	"This is the unsolved problem — three untrained agents. Zero success: the payload barely moves, progress is under 2%." (let it flail ~10 s)

Step	Press	What happens	What you say
2	3	QMIX window	"Same arena, same sensors — but this policy was learned . Watch it pull, rotate, and settle the payload in the zone." (let it finish one delivery, or press R for another run)
3	ESC → 4	MAPPO window	"A second algorithm on the identical task: much more cautious — slower, but zero collisions on every successful delivery." (~20 s)
4	ESC	back at menu	"Formal numbers: 30 fixed-seed episodes — QMIX 50% success and fastest deliveries, MAPPO ~27–30% but perfectly safe, APF baseline 100% only with privileged sensing."

Keys inside every window: **SPACE** pause · **R** new scenario · **+/-** speed · **ESC** quit.

*Tip: if a delivery looks chaotic on one scenario, just press **R** and say "let me show you another scenario" — you are re-rolling the obstacle layout, which is normal research practice, not cheating.*

4b. Who does what — splitting it across the team of 4

One laptop, one presenter at the keys — the other three speak when called. Assign these roles (pencil in your names):

Member	Role during the demo
A — Presenter	Runs the whole flow: gives the §1 pitch, presses 1→3→4→2, narrates each window live
B — Numbers & backup driver	Owns the §6 numbers table and the plots folder (press 8); answers every "what was the exact success rate?" moment; handles R resets if A is mid-sentence
C — Theory responder	Leads viva answers on CTDE, the reward function, curriculum, and why QMIX beats MAPPO (§7 Q1–Q7)
D — Systems responder	Leads answers on implementation and training: observation space, action grid, throughput, resumable runs, weaknesses & future work (§7 Q8–Q10); opens REPORT.pdf at the wrap (press 6)

Practical tips: - Everyone else answers questions — only member A touches the keyboard. - Agree beforehand on ONE sentence each of B, C, D will definitely say (e.g., C delivers the curriculum-ablation line; D delivers the ~570 env-steps/s training line) so all four visibly contribute — guides notice who speaks. - If sir asks "who did what?", answer with the module each member defended above; it should match how you divided the coding/report work.

5. THE FLOW — full 15–20 minute version (recommended)

Part A — Slides (8–10 min) — press 5 (the approved project PPT)

Walk the deck in its own slide order; whatever each slide shows, make sure these points get said along the way: 1. Problem & objective — keep it to ~1 minute; the live demo is the star. 2. **CTDE architecture** and the two MARL engines (MAPPO, QMIX) — the technical heart; spend 2–3 minutes here. 3. Curriculum learning — mention the ablation: **75% with vs 12–25% without curriculum**. 4. Method/expected-results slides — foreshadow: "these are the targets my trained agents actually meet; I'll show them live in a moment." 5. Close the deck with: "Now let me show you the actual trained system running live, sir." Then:

Part B — Live simulation (5–7 min) — ESC the slideshow, run `RUN_DEMO.bat`

Exactly the 4-step flow from §4, but slower: after QMIX (press **3**), press **R** two or three times, narrate one full delivery ("approach → drag → rotate → settle → bonus"), then show MAPPO (press **4**), and finish with the APF baseline (press **2**):

Step	Press	What you say
1	1	"Random policy — the baseline of failure. 0 of 30 episodes."
2	3	"QMIX — learned team, 50% success, delivers in ~220 steps on average when it succeeds. Each rover sees ONLY its own 9-beam sensor."
3	4	"MAPPO — ~27% success but 0 collisions on every success; tightest formation (rig stretch ~0.06 m). Cautious character."
4	2	"The classical APF controller: 100% success, near-straight paths — but it uses ground-truth positions of everything. Our rovers don't have that luxury."

Part C — Wrap (1 min) — press 6

Open REPORT.pdf, show the results section and the figures appendix, and say: "Full report, code, and evaluation logs are in the deliverables."

6. The formal numbers (say these confidently)

Same fixed-seed evaluation suite, full task (all obstacles active):

Policy	Success rate	Collisions (successes)	Steps to deliver	Path efficiency
Random	0%	—	never (600)	0.52
APF baseline	100%	2.8	80.6	1.02
MAPPO (best ckpt)	~27%	0.0	432	0.78
QMIX (best ckpt)	50%	47.5	224	0.38

One-line interpretations: - APF = optimal path but **privileged sensing** (ground-truth positions). - MAPPO = safe and formation-preserving, but slow and low success. - QMIX = the practical winner: fastest learned delivery, moderate collisions. - Curriculum ablation: **75% (training-time eval) vs 12–25%** with/without staged training; the strict 30-seed formal suite is harder than the training-time evaluation, so formal numbers are lower. - Training throughput ~570 env-steps/s on CPU; segmented resumable 8-min runs.

Honesty note about `final.pt`: QMIX training continued past its peak and the last checkpoint degraded (late-training collapse, a well-known RL instability). All demo/benchmark numbers therefore use `best.pt` — the checkpoint saved at the evaluation peak. Say this proactively if sir asks about training curves going down at the end: it shows you understand checkpoint hygiene.

7. Viva Q&A bank (read twice the night before)

Q1. Why multi-agent at all — why not one controller for all rovers? One centralized controller would need a joint action space of 27 discrete actions (3 rovers x 3 wheel levels) and full-state input. MARL factorizes the problem: each rover keeps a small local policy; the shared reward couples them, so cooperation emerges rather than being hand-programmed.

Q2. What is CTDE and why did you use it? Centralized Training, Decentralized Execution. During training a critic sees the full 27-dim global state (stable learning, handles the non-stationarity each agent causes for the others). At execution each rover acts on only its own 26-dim local observation — exactly what a real robot would have.

Q3. Why does QMIX beat MAPPO here? QMIX discretizes actions (3 wheel-speed levels) and trains a Q-function with replay and n-step returns — far more sample-efficient exploration. The monotonic mixer guarantees the joint value is consistent with per-agent utilities. MAPPO's continuous Beta policy explores slowly, so it converges to a safe-but-timid policy: 0 collisions, only 30% success. Same reward, two algorithms → two different "characters."

Q4. The APF baseline gets 100%. So why do RL at all? APF is privileged: it reads exact ground-truth positions of goal, payload, and every obstacle. Our rovers sense only 9 LiDAR beams of 3 m plus relative vectors. Also APF needs hand-tuned gains per arena; RL transfers by retraining. The research question was *learned* cooperation under realistic sensing.

Q5. What is the reward function exactly? One team reward shared by all rovers: +6 per metre of goal progress, −1.5 per metre of rig stretch (cohesion), −2 per rover collision contact, −4 per payload-obstacle contact, −0.02 per step (time), −1 out of bounds, +30 terminal bonus. Near the goal there is progressive shaping ($0.3 \cdot (1.5 - d)$) and a 0.15 hold bonus so it *settles* rather than orbits. Success = payload centre within 0.80 m, speed < 0.35 m/s, held 5 steps.

Q6. What does each rover observe? 26-dim local vector: 9 LiDAR beam distances (3 m range), relative goal and payload pose, teammate relative vectors, own velocity/heading.

Q7. What is the curriculum? Stage 0: empty arena → Stage 1: static obstacles → Stage 2: tighter static layout → Stage 3: adds a moving obstacle (which cannot camp on the goal). Ablation: 75% success with curriculum vs 12–25% training the full task directly — from-scratch agents get stuck in the hard maze.

Q8. Weaknesses? (Volunteer this — it builds trust) Stage-3 endgame: with the dynamic obstacle active, QMIX sometimes oscillates near the goal zone before settling (success-only collisions ~47 on the strict suite), and late training showed instability — the final checkpoint degraded, so we ship the best-evaluation checkpoint. Documented honestly in the report. Future work: finer endgame shaping, larger action grid, longer training, recurrent policies for partial observability.

Q9. How long did training take? Segmented resumable 8-minute runs (~570 env-steps/s on CPU), so hours spread across sessions — everything is reproducible via `python src/train.py --resume`.

Q10. What are the deliverables? Source (env, MAPPO, QMIX, APF, curriculum, evaluation), 30-episode formal benchmarks as JSON, 5 figures, 8-page report, presentation, and this live demo.

8. Troubleshooting (in the room)

Symptom	Fix
PyGame window opens behind other windows	Click it; or close extra windows before starting
A delivery looks messy	Press R for a new scenario; narrate the next one
Simulation feels slow	Press + to speed up; keep charger plugged in; "Best performance" power plan
Pressed ESC and everything vanished?	ESC closes only the sim window — the menu console is still open behind it
QMIX/MAPPO window shows an odd error	Press ESC, choose the other option (3↔4); both checkpoints are verified on disk
Professor asks for numbers you forgot	Press 8 (plots folder) or 6 (report) — or read the table from this guide

9. Backup plans

1. **PyGame will not start at all** → show the plots folder (option 8): learning curves, comparison bars, curriculum effect, trajectory plots tell the whole story without a live sim.
2. **Projector fails** → present the approved PPT (option 5) plus REPORT.pdf's results section and figures appendix from the laptop screen.
3. **Time cut to 2 minutes** → run the §4 short version: press 1, press 3, state the numbers table from memory.
4. **Worst case** → REPORT.pdf alone is a complete 8-page walkthrough with an embedded figures appendix.

10. Final checklist

- [] `RUN_DEMO.bat` double-clicks to menu
- [] Press 3 works and rovers deliver (R a few times)
- [] Press 4 works and is visibly cautious
- [] Press 5 opens the approved project PPT
- [] Press 6 opens REPORT.pdf
- [] Charger packed; power plan = Best performance
- [] §1 pitch and §6 numbers memorized
- [] This guide open on your phone as a cheat-sheet

Good luck — you built this, you can defend it.